

Lab Assignment: Defense Mechanisms Against Control Hijacking

Course: Computer & Network Security Lab **Topic:** Buffer Overflow Mitigations (Stack Canaries, NX/DEP, ASLR) **Duration:** 2 Hours **Environment:** Linux (Ubuntu/Debian/Kali recommended) with gcc and gdb.

1. Introduction

In previous labs, you may have exploited buffer overflows to hijack the control flow of a program (e.g., overwriting the Return Address). To combat this, modern operating systems and compilers implement several defense mechanisms.

In this lab, you will explore three critical defenses:

1. **Stack Canaries (SSP):** A secret value placed before the return address to detect buffer overflows.
2. **NX Bit (No-Execute) / DEP:** Marking the stack as non-executable to prevent shellcode execution.
3. **ASLR (Address Space Layout Randomization):** Randomizing memory locations to prevent attackers from predicting addresses.

2. Prerequisites

Ensure you have the following tools installed:

```
sudo apt-get update
sudo apt-get install build-essential gdb gcc
# Optional but recommended for easy checking:
sudo apt-get install checksec
```

3. Lab Exercises

Part A: Stack Canaries (Stack Smashing Protector)

Theory: A "Canary" is a random value placed on the stack just before the return address.

Before a function returns, the program checks if the canary is still intact. If a buffer overflow has occurred, the canary will likely be overwritten/corrupted, and the program will deliberately crash (** stack smashing detected **) to prevent the attacker from redirecting control flow.

Step 1: Create the Vulnerable Program Save the following code as vuln.c. It contains a classic buffer overflow vulnerability.

```
// vuln.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```

void secret_function() {
    printf("Congratulations! You hijacked the control flow!\n");
    exit(0);
}

void vulnerable_function(char *str) {
    char buffer[16]; // Small buffer
    // Vulnerability: strcpy does not check bounds
    strcpy(buffer, str);
    printf("Input copied: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input_string>\n", argv[0]);
        return 1;
    }
    vulnerable_function(argv[1]);
    return 0;
}

```

Step 2: Compile with and without Canaries Compile two versions of the binary: one unprotected and one protected.

1. **Disable Canary (Unprotected):**

```
gcc -fno-stack-protector -z execstack -no-pie -o vuln_unprotected vuln.c
```

2. **Enable Canary (Protected):**

```
gcc -fstack-protector-all -z execstack -no-pie -o vuln_canary vuln.c
```

Step 3: Test the Defense

1. Run the **unprotected** binary with a long string (more than 16 bytes + padding).

```
./vuln_unprotected $(python3 -c "print('A'*40)")
```

Observation: You should see a Segmentation fault. This is an uncontrolled crash (or potential exploit).

2. Run the **protected** binary with the same input.

```
./vuln_canary $(python3 -c "print('A'*40)")
```

Observation: You should see an error message like: ***** stack smashing detected ***: terminated.** The program caught the attack and shut down safely.

Step 4: Analyze the Assembly Use gdb to see the canary in action.

```
gdb -q ./vuln_canary
```

Inside GDB:

```
disas vulnerable_function
```

- **Look for the Prologue:** You will see code reading from `%fs:0x28` (on 64-bit systems) and placing it onto the stack. This is the canary setup.

- **Look for the Epilogue:** Before ret, you will see a xor instruction comparing the stack value with %fs:0x28. If they are not equal, it calls __stack_chk_fail.

Part B: NX Bit (No-Execute) / DEP

Theory: The NX bit (Data Execution Prevention or DEP in Windows) marks certain areas of memory (like the stack and heap) as "non-executable." Even if an attacker injects shellcode into the buffer, the CPU will refuse to execute it.

Step 1: Compile with and without NX Using the same vuln.c from Part A.

1. **Disable NX (Executable Stack):**

```
gcc -fno-stack-protector -z execstack -o vuln_execstack vuln.c
```

2. **Enable NX (Non-Executable Stack):**

```
gcc -fno-stack-protector -z noexecstack -o vuln_nx vuln.c
```

(Note: Modern GCC enables NX by default, so explicit -z noexecstack is standard behavior, but we specify it for clarity).

Step 2: Verify Permissions You don't need to run an exploit to see this; you can examine the binary headers.

Run the following command on both binaries:

```
readelf -l vuln_execstack | grep GNU_STACK
readelf -l vuln_nx | grep GNU_STACK
```

- **Task:** Look at the Flg (Flags) column.
 - RWE means Read, Write, **Execute**. (Vulnerable)
 - RW means Read, Write. (Safe - code cannot run here).

Part C: Address Space Layout Randomization (ASLR)

Theory: ASLR randomizes the memory locations of program components (stack, heap, libraries) every time the program is run. This makes it difficult for an attacker to hardcode a return address (e.g., the address of system()) in their exploit.

Step 1: Create an Address Printer Create a new file aslr_test.c:

```
// aslr_test.c
#include <stdio.h>
#include <stdlib.h>

void main() {
    int stack_var = 0;
    int *heap_var = malloc(4);

    printf("Stack Address: %p\n", &stack_var);
    printf("Heap Address: %p\n", heap_var);
    printf("Main Address: %p\n", main);

    free(heap_var);
}
```

Compile it:

```
gcc -o aslr_check aslr_test.c
```

Step 2: Test with ASLR OFF Linux controls ASLR via the file `/proc/sys/kernel/randomize_va_space`.

- 0 = Off
- 2 = Full Randomization

<!-- end list -->

1. Turn ASLR **OFF** (requires root):

```
sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

2. Run the program 3 times:

```
./aslr_check  
./aslr_check  
./aslr_check
```

Observation: The addresses should be **identical** every time. This makes exploitation easy.

Step 3: Test with ASLR ON

1. Turn ASLR **ON**:

```
sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```

2. Run the program 3 times:

```
./aslr_check  
./aslr_check  
./aslr_check
```

Observation: The Stack and Heap addresses should change every time.

4. Lab Report Questions

Prepare a report answering the following:

1. **Stack Canaries:**

- Paste the assembly code from Part A (Task 4) that shows the canary being compared before the function returns.
- Why does the canary protection fail if the attacker can read memory (information leak) before overflowing the buffer?

2. **NX/DEP:**

- If the stack is non-executable, attackers cannot run shellcode placed on the stack. What advanced technique do attackers use to bypass this? (Hint: It involves reusing existing code, often called ROP).

3. **ASLR:**

- In Part C, did the "Main Address" change when ASLR was enabled? If not, investigate what "PIE" (Position Independent Executable) is and how it relates to ASLR.

4. **Synthesis:**

- Which of these three defenses would most likely prevent a "Return-to-Libc" attack? Explain why.

End of Assignment